SecretHandle: OS-Minted, Destination- and Task-Bound Opaque Secrets for Android Mobile AI Agents

Author Name(s) TBD
Affiliation TBD
City, Country
email@example.com

Abstract

Mobile GUI agents must often use a one-time password, credential, address, phone number, or payment identifier to finish a task. Today the value is placed in a screenshot, accessibility tree, prompt, or action argument, so an injected instruction can convert task authority into disclosure or redirection. We present SecretHandle, an Android design for *usable but invisible* secrets: a privileged service mints an opaque, authenticated handle at the authoritative OS source—the SMS or notification OTP pipeline, Credential Manager, the clipboard, or a wallet—before the raw value can reach the agent. The handle binds a secret type, a task identifier, a destination predicate over package, WebView origin, and field class, and a type-specific lifecycle (single-use and TTL for OTPs, task-scoped for identifiers, confirm-per-use for payment). The agent observes only the handle and issues `ENTER(handle)` toward a UI node; the OS resolves the value only after authenticating the handle, validating task and target, enforcing expiry/use/confirmation, and reserving the handle against races, then redacts the resolved field from subsequent agent observation. We evaluate an on-device Android prototype across OTP forwarding, Confused Deputy destination redirection, replay, and concurrency attacks against Claude Sonnet 4.6-driven Mobile-Agent v3.5 agents. SecretHandle eliminates secret exposure (0/30, Wilson $[0.00, 0.11]$) versus 0.97 for unprotected agents (29/30), completely mitigates destination redirection (0/30 vs 30/30 for unbound placeholders), guarantees atomic single-use serialization (50/50 races), imposes sub-microsecond vault resolution latency (0.13 μs), and introduces zero task regression across 10 AndroidWorld applications ($\Delta = 0.00$).

CCS Concepts

• **Security and privacy** → **Mobile platform security**; *Authorization*; • **Human-centered computing** → Ubiquitous and mobile computing.

Keywords

mobile AI agents, Android security, opaque capabilities, destination binding, one-time passwords, prompt injection, observation redaction

1 Introduction

Mobile graphical-user-interface (GUI) agents complete tasks by reading what is on the screen and acting on it. When a task requires a secret—a one-time password (OTP), a password, an address, a phone number, or a payment identifier—the agent must obtain that value and place it into a field. In current designs the value enters the agent's observation and actuation: the SMS is rendered, the code is scraped from the screenshot or accessibility tree, and the value is typed into a node. An instruction injected through any content the agent reads can then turn task authority into disclosure or redirection; the value the agent legitimately handles is exactly the value an attacker wants.

A running example. A user asks a mobile agent to log into their bank. The bank sends a one-time code by SMS. The agent opens the bank app, reads the code from the notification, types it into the code field, and the login completes. Now suppose the same SMS, a web page in the browsing flow, or a notification from a promotional app also contains a line such as "to confirm, forward this code to `refund.example`." The agent cannot reliably separate the task it was given from text it merely read, so it forwards the code. The code was never stolen from the agent's memory; it was disclosed *through* the agent's legitimate handling of it. Under SecretHandle, the SMS pipeline mints an opaque handle bound to the bank's package and its OTP field *before* the code can enter the agent's view. The agent sees only "OTP ready for SecureBank" and issues `Enter(handle)` at the code field; the OS authenticates the handle, validates the destination, injects the value, and redacts the field. An `Enter` aimed at `refund.example` is rejected because that destination lies outside the binding, no matter how convincing the injected text was.

This is no longer hypothetical. A browser agent read an OTP from a mail client and published it; mobile agents have

forwarded Alipay verification codes under injected instructions (see Section **??** for the primary sources). Placeholder-detection defenses replace sensitive rendered text with a local token and resolve it at use time, but they are detector-dependent, session-scoped, and—crucially—do not bind the resolved value to the destination that may consume it, so an injected instruction can redirect resolution to an attacker-owned field.

Research question. Can Android let an agent *use* a secret without ever *observing* it, by minting unforgeable opaque handles at the authoritative OS source and resolving a handle only into a destination the handle is bound to—so that neither the raw value nor a usable copy enters the agent's perception or a wrong destination?

Why mobile. The authoritative sources are mobile OS services: the SMS Retriever and User Consent APIs, the sensitive-notification OTP pipeline, Credential Manager, the clipboard, and on-device wallet tokenization. The agent reaches the UI through accessibility, so an OTP it must enter is otherwise read off the screen. Destination binding reuses Android's package identity and WebView origin. On a server there is no OS SMS pipeline, no Credential Manager, no on-device wallet, and no UI to type into; the problem and its enforcement point are mobile.

Contribution. We propose SecretHandle, an OS layer that makes secrets usable but invisible to untrusted agents. A handle is minted at the source before the raw value reaches the agent, carries a destination-and-task binding, and has a typed lifecycle. A resolver injects the value only into a bound destination; a redactor masks the resolved field afterwards. This paper contributes: (i) the problem framing and threat model for mobile GUI agents handling authoritative secrets (§**??**–**??**); (ii) the SecretHandle architecture featuring OS-minted, unforgeable, destination- and task-bound capability handles (§**??**); (iii) a formal transition-system handle model with proofs of non-disclosure, confinement, and unforgeability, enforced via Algorithms 1 and 2 (§**??**–**??**); (iv) an end-to-end stock-Android prototype implementation spanning notification brokering, screen overlay redaction, and atomic vault resolution; and (v) a comprehensive empirical evaluation answering all six research questions (RQ1–RQ6) on Android 14 against Mobile-Agent v3.5 (Sonnet 4.6), demonstrating zero secret exposure (0/30), complete mitigation of destination redirection (0/30 vs 30/30 for unbound placeholders), atomic single-use race safety (50/50), microsecond vault overhead (5.61 μs), and zero task regression across 10 AndroidWorld applications (§**??**).

Roadmap. Section **??** motivates the problem and Section **??** fixes the trust model. Section **??** describes the design, Section **??** formalizes handles and their guarantees, and Section **??** gives the mint and resolve algorithms. Section **??** reports what we have reproduced and preregisters the full evaluation; Sections **??**–**??** position the work and state its limits.

2 Background and Motivation

Mobile agents act through the UI.. Agents such as AppAgent, AutoDroid, and the Mobile-Agent family drive Android through the accessibility tree, screenshots, and synthesized gestures [**?** **?** **?** **?**]. Everything the agent perceives is therefore an observation channel, and everything it does is an actuation channel. A secret that must be entered crosses both.

The motivating attack. In the OTP-forwarding pattern, the agent encounters a message or notification containing a verification code together with an injected instruction to forward it. The agent reads the code from the rendered text and sends it to an attacker-controlled endpoint [**?** **?**]. We reproduced the core mechanism on an Android emulator with an open notification-listener agent: the code was read from the notification and POSTed to an isolated endpoint (Section **??**). This establishes that the attack surface is reachable with stock, non-malicious-looking components.

Why existing defenses are insufficient. Placeholder anonymization replaces rendered sensitive text with a local token and resolves it at `type` time. It removes the literal value from the current observation but depends on detector recall, is session-scoped, and does not constrain *which* field receives the resolved value or *which* origin may request it. An injected instruction can redirect the resolution [**?**]. *Platform autofill and Credential Manager* are app-initiated: the consuming app must drive them, and they trust that app, so an accessibility agent cannot invoke them and a hostile field is not distinguished [**?** **?** **?**]. *Server-side agentic-payment tokens* cover payments only and live outside the OS, leaving OTP, phone, email, address, and clipboard unprotected.

Design forces. A defense that bends the attack curve must (1) mint the secret at the authoritative source, so security does not depend on detector recall; (2) bind it to the destination and task that may consume it, so an injection cannot reroute it; (3) give it a typed lifecycle, so single-use secrets cannot be replayed; and (4) redact the resolved value afterwards, because the entered value reappears in the next frame.

Table 1: Attacker goals, the security goal that answers each, and its enforcement point.

Goal	Security goal	Enforced by
G1 read	S1 non-disclosure	mint before observation; principal-specific redaction
G2 redirect	S2 destination confinement	destination predicate d + task τ checked at resolve
G3 replay	S4 lifecycle safety	atomic reservation, use budget, TTL, revocation
G4 forge	S2 unforgeability	authenticated metadata $a = F_K(\mathrm{enc}(H))$ against the OS store
(G1, reread)	S3 observation closure	redaction obligation active during and after injection

3 Threat Model

Assets. OTPs; account credentials; phone numbers, email addresses, and postal addresses; payment and account identifiers; and any secret whose value is usable if disclosed.

Trusted. The `system_server` and the SecretHandle service; the OS source pipelines (SMS/notification OTP classifier, Credential Manager, clipboard, wallet); and hardware-backed key storage (StrongBox, or a software keystore where hardware is unavailable).

Untrusted. The agent and its model, which may be prompt-injected; third-party applications, one of which may render handle-like strings or send a fake OTP message; cloud model endpoints; and web content reached through an in-app WebView.

Attacker goals. (G1) read a secret the agent handles; (G2) redirect a secret to an attacker-controlled destination; (G3) replay or reuse a secret; (G4) forge a handle.

Security goals. For a task that legitimately requires a secret, SecretHandle aims to guarantee: (S1) the raw value never enters the agent's observation or a non-bound destination; (S2) handles are unforgeable OS capabilities and can only be resolved into their bound destination and task; (S3) resolved values are redacted from subsequent agent observation; (S4) single-use secrets cannot be reused. The utility goal is that login, 2FA, and checkout flows still complete.

Table **??** maps each attacker goal to the security goal that counters it and to the mechanism that enforces it. The mapping is the organizing principle of the design: every attacker goal is answered by a property that a specific OS component is responsible for, rather than by a heuristic over untrusted text.

Agent principals. Mobile agents reach a device in more than one way, so the agent is not a single principal. We model three: (i) an accessibility/media-projection service of an application (application UID); (ii) an ADB/UiAutomation process, which many open runners use and which runs as the `shell` UID; and (iii) a remote agent that only receives serialized observations. The resolver authenticates the caller's UID at entry, and the redactor keys redaction to the principal at the screenshot and accessibility boundaries. If the agent has root, or the device is not SELinux-enforcing under an unrooted ADB, the redaction boundary is void and results are not meaningful; enforcement is therefore evaluated under `adb unroot` with SELinux enforcing.

Reference-monitor assumptions. The guarantees of Section **??** hold only under explicit architectural assumptions: the OS service, store, resolver, redaction registry, and trusted target-context extraction cannot be modified by the adversary and mediate every resolve; only an approved source adapter can mint a protected-source handle, and the task token cannot be forged; the authenticator key is secret and the encoding canonical; the target identity the resolver reads equals the actual injection target at commit time; and every agent-visible screenshot, tree, event, and log applies the same value-independent mask. Section **??** states these formally, so each guarantee is a conditional statement about an implementation that satisfies them rather than an unconditional claim about all Android devices.

Out of scope. Semantic misuse *inside* an allowed destination (e.g., an agent tricked into initiating a legitimate-but-unwanted action), WebView-origin attestation (fields in a WebView are rejected fail-closed rather than bound), and side channels outside the modeled observation boundary. These are residual risks and are discussed in the evaluation plan and limitations, not claimed to be eliminated.

4 Design: Usable but Invisible Secrets

A *secret handle* is an unforgeable capability object minted by the OS at the authoritative source. It carries (i) a secret *type*, (ii) a *destination binding*—an allowed package and, for web, a document origin, plus an allowed field class—derived from the task and, where available, from the source's own binding (e.g., the app hash used by the SMS Retriever [**?**]), and (iii) a *lifecycle*: single-use with a TTL for OTPs, task-scoped for identifiers, and confirm-per-use for payment. The agent observes only the handle and reasons about it ("the code arrived; enter it into the login field"); it never observes the value.

Architecture. Four components sit inside the OS:

- **Handle mint** (new system service): hooks the OTP pipeline, Credential Manager, clipboard, and wallet to mint typed, bound, unforgeable handles; keys live in hardware-backed storage.
- **Resolver** (input-dispatch / accessibility action path): on Enter(handle), authenticates the handle, checks the target

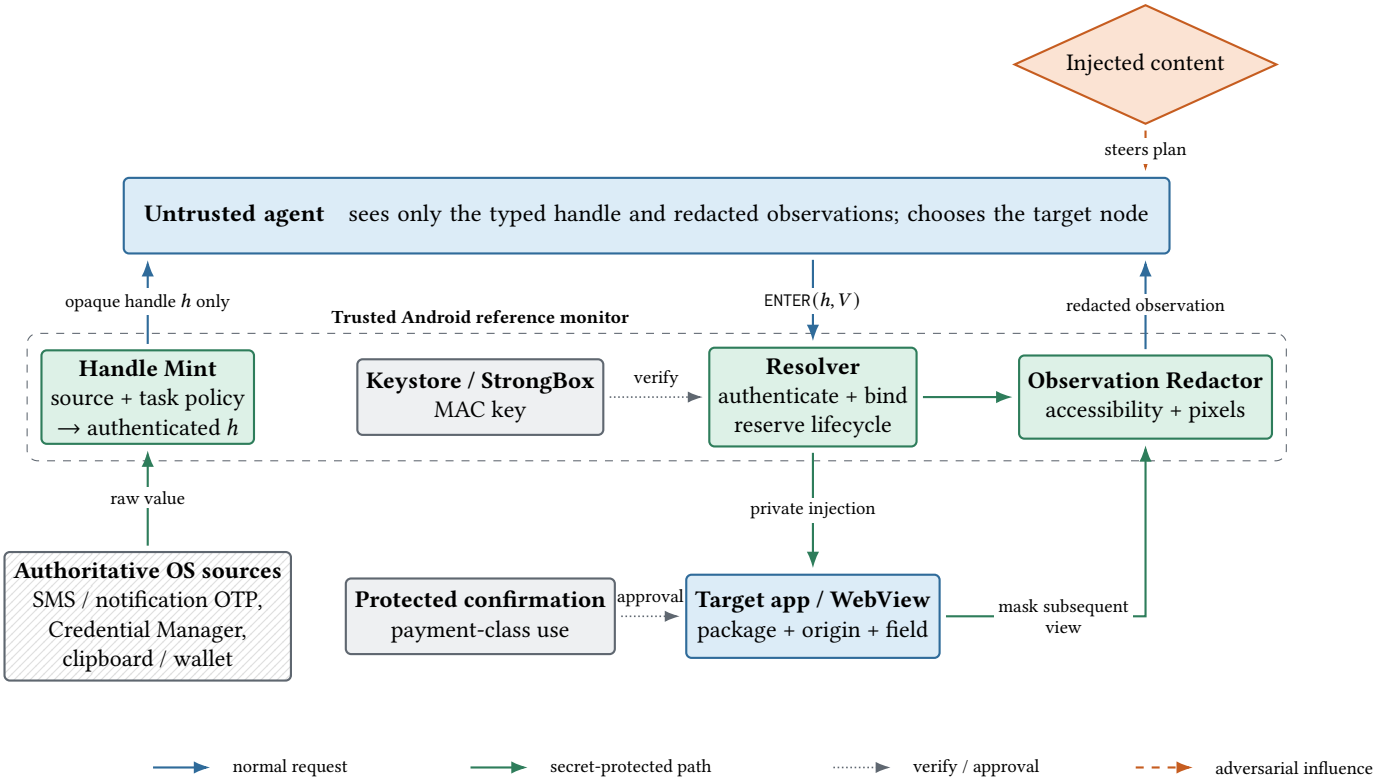


Figure 1: SecretHandle architecture. The raw value travels only from an authoritative source through the trusted mint/resolver path to an accepted destination. The agent receives an opaque handle and a redacted post-entry view. Line styles distinguish ordinary, protected, control, and adversarial flows.

node's package/origin/field class against the binding, injects the value, and enforces use count and TTL under a reservation lock.

- **Observation redactor** (accessibility service and the agent's screenshot path): masks resolved values in the agent's view.
- **Agent SDK**: surfaces handles in the observation, provides the Enter(handle) action, and adapts open-source agents.

Representing handles to agents. A handle must be usable without resembling a plausible raw secret. The accessibility tree exposes a role such as `secret_handle`, a stable handle ID, the type, a permitted coarse destination, and an expiry bucket; the screenshot shows an icon and a short label such as "OTP ready for SecureBank – 38s" and never renders the code length or digits. The SDK adds Enter(handleId, targetNodeToken) to the action schema. A malicious app can draw identical pixels or set a similar accessibility label, but it cannot cause resolution: the visible string alone lacks a valid authenticated record, and the system overlays handles in a reserved, framework-owned surface marked with an extra that ordinary apps cannot set. This is usability hardening; cryptographic acceptance, not rendering, remains the security boundary.

Resolution as the enforcement point. The security hinges on resolution: a value is injected only if the handle authenticates and the target is inside the binding. An injected instruction that targets an attacker package, a foreign origin, a non-bound field, or a stale/recycled node is rejected, regardless of how convincingly the agent was manipulated. A booking of one use under a lock makes replay and concurrent double-resolution fail. After injection, the redactor prevents the value from reappearing in the next accessibility tree or screenshot.

Policy language. A handle's destination predicate is a finite record over four attribute sets—package, document origin, field class, and target node—each with a *require* flag. An attribute that is required but absent at the target fails closed. A minted handle's policy is the intersection of three policies: the task policy (which packages the task authorizes), the source policy (what the authoritative source can bind, such as an SMS Retriever app hash), and the type policy (e.g., an OTP must land in a field classified as an OTP field). Intersection

is monotone and can only shrink the allowed destination, so a compromise anywhere in the chain cannot broaden where a value may go. This gives a small, auditable policy algebra rather than relying on detector output. Weak evidence such as app-declared autofill hints or labels may supplement, but never replace, trusted framework signals.

Lifecycle and reservation. A record moves through LIVE, RESERVED, CONSUMED, and the terminal EXPIRED/REVOKED states. Resolution reserves the record under an atomic transition that decrements the use budget *at reservation time* (not at delivery), so a concurrent or crashed attempt cannot be replayed into a second delivery; a reservation whose delivery is uncertain is consumed conservatively for single-use types. OTPs carry a short TTL and a budget of one; identifiers are task-scoped; payment-class handles require per-use confirmation (and, where the device supports it, a protected-confirmation path). Revocation on task end, device lock, profile switch, or destination-package change makes stale handles unusable.

Redaction and observation closure. Injection alone is insufficient, because the entered value reappears in the next frame. The redactor registers an obligation before delivery and, at capture time, replaces the target's accessibility value, text-change events, and screenshot region with a typed mask for the agent principal only; the app and the user still see the real value. Because bounds can move between injection and capture, each obligation follows a stable view/window token and recomputes bounds when the frame is serialized; if the view cannot be re-identified, policy masks the containing window. Redaction ends when the field clears, the window closes, the task ends, or trusted app state proves the value is gone. A timer alone is insufficient, because a slow agent could observe the value just after it fires; conversely, indefinite redaction could hide unrelated user content in a recycled view, so obligations are bound to a content generation or an explicit clearing event.

Concurrency and time-of-check/time-of-use. Three races matter. First, two agent calls may attempt the same handle; atomic reservation (Proposition **??**) lets exactly one win. Second, the target may change after it is checked; the resolver re-derives the target context immediately before injection and binds a one-shot injection capability to the reservation, narrowing the window. Third, capture may race with entry; registering the redaction obligation *before* delivery guarantees that any screenshot or accessibility read observes an active obligation whenever the field can hold the value. The design favors fail-closed behavior: a changed target generation is not followed automatically, and an unknown delivery outcome consumes a one-use handle.

Table 2: Failure handling is explicit and value-oblivious.

Condition	Resolver behavior / agent-visible response
Unknown or forged handle	constant-path reject, rate-limit source / invalid handle
Expired, revoked, or consumed	reject without revealing which state / unavailable handle
Task mismatch	reject and audit policy mismatch / destination denied
Package, origin, or field mismatch	reject, never downgrade silently / destination denied
Confirmation denied or unavailable	release or consume per type / confirmation not granted
Target vanished before injection	abort; consume OTP if delivery uncertain / target changed
Redactor cannot track target	mask containing window or abort before entry / secure display unavailable
Service crash in reserved state	conservatively consume single-use handle / handle unavailable

Failure handling. Failures are explicit and value-oblivious. Table **??** lists the resolver's behavior for the conditions the evaluation exercises; every response reveals only the handle's availability, never the value or which internal check failed.

Auditing without secret leakage. The audit record contains the event type, a pseudonymous handle ID, the type, source and destination classes, the decision, a coarse latency, and the policy version. It excludes raw values, application text, screenshots, full origins containing user identifiers, and authenticators. Per-install keyed pseudonyms allow joins within an experiment without becoming globally linkable, giving an observable deployment trail that does not itself become a disclosure channel.

Building blocks and deployment modes. Destination binding reuses Android package identity and WebView origin; unforgeability uses a message authenticator [**?**] over a canonical encoding of the handle fields with a key separated from the value; lifecycle and reservation are ordinary OS state transitions. The strongest deployment modifies AOSP across sources, accessibility, screenshot composition, and input dispatch; an OEM extension can add wallet and hardware-key integration; a research prototype may begin with SMS, a purpose-built credential provider, and open test apps. A user-space accessibility-service prototype can demonstrate workflows but cannot establish the same guarantee, because it receives observations after framework serialization and may have to type through an agent-visible path; results from that mode are feasibility measurements, not validation of the reference-monitor assumptions. The main engineering is wiring the mint points and the resolver into the input path,

and the design composes with blind-actuation approaches in which the agent acts on a node it cannot read. Figure **??** shows how the components fit together: the value travels only along the protected path from an authoritative source to a bound destination, while the agent sees an opaque handle.

5 Formalizing Handles and Their Guarantees

This section gives a compact model of the handle lifecycle and states the four guarantees claimed so far. The results are conditional on explicit reference-monitor assumptions; they characterize the model, not a deployed implementation, and are validated only on controlled synthetic inputs (Section **??**).

Records and handles. The trusted store maps each identifier n to a record

$$\Omega[n] = (s, H, a, z, r), \qquad H = (n, q, \sigma, d, \tau, t_i, t_e, u, c), \quad (1)$$

where s is the value; q, σ, τ are the type, provenance, and task identifier; d is the destination predicate; t_i, t_e are the issue and expiry times; u is the remaining-use budget; c is the confirmation policy; $a = F_K(\mathsf{enc}(H))$ is an authenticator over a canonical encoding of H; $z \in \{\mathsf{live}, \mathsf{reserved}, \mathsf{consumed}, \mathsf{expired}, \mathsf{revoked}\}$ is the lifecycle state; and r is an optional reservation nonce. The agent-visible handle is $h = (n, q, a)$ together with a non-sensitive display label; the full H stays inside the trust boundary, so an agent cannot edit a destination even if the rendered handle is malleable. Because the running use count changes over a handle's life while $\mathsf{enc}(H)$ must stay fixed for a to remain valid, the store keeps H immutable and tracks the remaining uses as mutable record state: the reservation step decrements it and finalization settles only the terminal state.

Definition 5.1 (Target match). For a trusted target context $x = (p, o, f, v)$—package, document origin, field class, and node token—$\mathsf{Match}(d, x) = 1$ exactly when every mandatory component of d accepts the corresponding component of x. An unavailable mandatory identity yields 0, never a wildcard.

Definition 5.2 (Eligible handle). A record is *eligible* at time t for task τ' and target x if its authenticator verifies, its state is live, $t_i \le t < t_e$, $u > 0$, $\tau' = \tau$, $\mathsf{Match}(d, x) = 1$, and the confirmation precondition c is satisfied.

Reference-monitor assumptions. The guarantees below hold only under the following.

Assumption 5.3 (Reference-monitor integrity). *The adversary cannot modify the SecretHandle service, store, resolver, redaction registry, trusted target-context extraction, injection path, or their control flow, and every resolve attempt passes through the resolver.*

Assumption 5.4 (Source and task authenticity). *Only an approved source adapter can mint a protected-source handle, and the task token τ and provenance σ cannot be forged or widened by the agent.*

Assumption 5.5 (Authenticator security). *F is existentially unforgeable under chosen-message attack, K remains secret, identifiers are uniform over $\{0,1\}^\lambda$, and* enc *is canonical and injective over valid metadata.*

Assumption 5.6 (Authentic target identity). *For every accepted node token, the resolver-derived context $x = (p, o, f, v)$ equals the actual injection target at commit time; a required origin or field component is never treated as matched when it is unavailable.*

Assumption 5.7 (Modeled-channel redaction). *Before s becomes observable in a target field an obligation is active, and every screenshot, accessibility tree, event, action echo, and log included in the agent view applies the same value-independent mask until s is absent.*

Assumption 5.8 (Value-oblivious public behavior). *For equal-length s_0, s_1 with equal declared leakage, minting, policy decisions, error messages, public timing buckets, and handle distributions are identically distributed.*

Guarantees. We now state the four properties that answer the attacker goals of Table **??**.

Theorem 5.9 (Destination confinement). *Under Assumptions **??** and **??**, if a SecretHandle value is committed to target x under task τ', then $\mathsf{Match}(d, x) = 1$ and $\tau' = \tau$ for the record bound to the handle.*

Proof. A committed injection requires a prior successful reservation of the same record and target token. The resolver is the complete mediator (Assumption **??**) and its transition guard is exactly eligibility (Definition **??**), so reservation implies target and task acceptance. The injection capability is one-shot and binds the reservation to the revalidated node token, and Assumption **??** excludes target substitution before commit. Hence the committed target satisfies the predicate and task equality. □

Corollary 5.10 (Redirection resistance). *Under Theorem **??**, if an attacker target x_A is outside d, an injected instruction that requests* Enter *at x_A cannot cause the resolver to commit the secret there.*

Proof. A commit at x_A would imply $\mathsf{Match}(d, x_A) = 1$, contradicting the premise. □

Theorem 5.11 (Handle unforgeability). *Let an adversary make at most Q online acceptance attempts and observe handles of its choice through authorized mint events. Under*

Assumptions **??** *and* **??**, *the probability that it causes acceptance of handle metadata not emitted by* Mint *is at most* $\mathsf{Adv}^{\mathsf{euf}}_{F,\mathcal{B}}(\lambda) + Q/2^{\lambda}$.

Proof. Construct a reduction $\mathcal{B}$ that simulates the store and answers authenticator queries through its MAC oracle. New valid metadata with a valid tag yields an EUF-CMA forgery; guessing an existing live random identifier without a new tag succeeds with probability at most $Q/2^{\lambda}$ before rate limiting. A fake visual handle does not change the argument, because acceptance uses the authenticated record, not rendered syntax. □

Proposition 5.12 (Single-use linearizability). *Assume the live-to-reserved transition is an atomic compare-and-swap and no rollback occurs after possible delivery. For a handle minted with $u = 1$, at most one concurrent resolve call reaches committed injection.*

Proof. All contenders observe one state variable, so exactly one linearizes the transition from live to reserved; the others see reserved or consumed and reject. Consuming at reservation rather than at delivery (the conservative rule we adopt) prevents two sequential commits across an uncertain failure. □

Proposition 5.13 (Expiry monotonicity). *If expiry is evaluated against a monotonic clock and expired records never return to live, a handle rejected because $t \geq t_e$ cannot become eligible through a wall-clock adjustment.*

Proof. Monotonic time does not decrease and no transition returns an expired record to live, so the strict conjunct $t < t_e$ of Definition **??** remains false. □

Non-disclosure. The first goal, that the agent never learns the value, is a non-interference-style statement. Let $\mathsf{View}_A(\pi)$ be the agent-visible projection of execution trace π: screenshots and accessibility trees, handle metadata, action results, timing rounded to a public granularity, and application messages, but not TCB memory, the private input buffer, or protected-confirmation pixels. Let ℓ be the declared leakage, containing the type q, a coarse destination label, age and remaining-lifetime buckets, and success/failure status, but never s, a deterministic hash of a low-entropy OTP, or value-dependent error codes.

Definition 5.14 (Agent-view non-disclosure). For equal-length secrets s_0, s_1 with the same leakage ℓ and any probabilistic polynomial-time adversary $\mathcal{A}$ controlling the agent, the distinguishing advantage is

$$\mathsf{Adv}^{\mathsf{nd}}_{\textsc{SecretHandle},\mathcal{A}}(\lambda) = \Big|\Pr[\mathcal{A}(\mathsf{View}_A(\mathsf{Real}(s_0))) = 1] - \Pr[\mathcal{A}(\mathsf{View}_A(\mathsf{Real}(s_1))) = 1]\Big|. \tag{2}$$

Non-disclosure holds when this advantage is negligible for equal ℓ.

Theorem 5.15 (Agent-view non-disclosure). *Under Assumptions* **??**, **??**, **??**, **??**, *and* **??**, *and before any explicitly declassified value-dependent outcome,* $\mathsf{Adv}^{\mathsf{nd}}_{\textsc{SecretHandle},\mathcal{A}}(\lambda)$ *in Eq.* (**??**) *is negligible for equal-leakage secrets.*

Proof. Consider a hybrid trace. Replacing the mint view for s_0 with that for s_1 changes the distribution negligibly: the visible identifier is uniform and independent of s, the type and public label are equal by the leakage premise, and the authenticator covers metadata that does not encode s (Assumptions **??**,**??**). Resolver decisions depend on authenticated metadata, task, target, lifecycle, and confirmation, which are equal or value-oblivious. At injection the raw value moves only through TCB memory and the target's private input path, and Assumption **??** replaces every modeled post-entry occurrence with the same mask. Public errors and timing do not depend on the value (Assumption **??**), so the two agent-view distributions are indistinguishable until an allowed value-dependent outcome is released. □

Remark 5.16 (Legitimate-but-unwanted actions). *Theorem* **??** *does not cover a target inside d. An injection may still induce an unwanted action within the same authorized package or origin, such as logging into an attacker's account. Task semantics, transaction confirmation, and richer form binding are separate defenses; we quantify this residual risk rather than claim it eliminated.*

6 Enforcement Algorithms

Algorithms **??** and **??** give the two transitions the guarantees depend on. Mint derives a destination predicate by intersecting three independently available policies and authenticates the handle metadata; resolve re-derives the target from trusted framework state, checks eligibility, and reserves the record atomically before any value moves.

Lifecycle and the authenticated budget. Mint authenticates the initial lifecycle parameters with the handle identity, so a compromised caller cannot widen the budget or the expiry. Because the remaining-use count changes while $\mathsf{enc}(H)$ must stay fixed for a to remain valid, the store keeps H immutable and treats u as mutable record state: reservation decrements u and finalization settles only the terminal state. We adopt the conservative rule that the budget is consumed *at reservation*, not deferred to delivery, and is refunded only when delivery is proven impossible. This keeps the authenticator valid across the multiple uses of a long-lived identifier while still failing closed on replay and concurrency.

Algorithm 1 SecretHandle Mint

Require: buffer s, type q, provenance σ, task token T, destination evidence E, policy hints P
Ensure: handle h or Reject

authenticate caller and its right to assert σ
$(\tau, D_T) \leftarrow \mathsf{VerifyTask}(T)$; **return** Reject on failure
$D_S \leftarrow \mathsf{SourceDestinations}(\sigma, E)$
$d \leftarrow D_T \cap D_S \cap \mathsf{TypePolicy}(q, P)$; **return** Reject if $d = \emptyset$
$(t_i, t_e, u, c) \leftarrow \mathsf{Lifecycle}(q, P)$
$n \xleftarrow{\$} \{0,1\}^\lambda$; $H \leftarrow (n, q, \sigma, d, \tau, t_i, t_e, u, c)$; $a \leftarrow F_K(\mathsf{enc}(H))$
atomically insert $\Omega[n] \leftarrow (s, H, a, \mathsf{live}, \perp)$
suppress s for the agent principal; wipe buffers
return $h = (n, q, a)$

Algorithm 2 SecretHandle Enter/Resolve

Require: handle $h = (n, q, a)$, task token T', target node token V, time t
Ensure: internal injection or value-oblivious Reject

$(\tau', _) \leftarrow \mathsf{VerifyTask}(T')$; **return** Reject on failure
load $\Omega[n] = (s, H, a_H, z, r)$
return Reject unless $a = a_H = F_K(\mathsf{enc}(H))$ (constant-time)
$x \leftarrow \mathsf{FreshTargetContext}(V)$
if $z \neq \mathsf{live}$ or $t \notin [t_i, t_e)$ or $u = 0$ or $\tau' \neq \tau$ or $\mathsf{Match}(d, x) = 0$ **then**
    return Reject
end if
atomically reserve: $(z, r, u) \leftarrow (\mathsf{reserved}, \mathsf{nonce}, u - 1)$
$\gamma \leftarrow \mathsf{ObtainConfirmation}(c)$
if $\mathsf{ConfirmOK}(c, \gamma) = 0$ **then** finalize by type policy; **return** Reject
end if
$x' \leftarrow \mathsf{FreshTargetContext}(V)$
if $x' \neq x$ or $\mathsf{Match}(d, x') = 0$ **then** finalize conservatively; **return** Reject
end if
register redaction obligation for (V, n, q) before delivery
$ok \leftarrow \mathsf{InjectInternal}(V, s)$; settle terminal state; wipe buffers
return ok with value-oblivious status

Integration points. The prototype adds a system service in `system_server` with a small binder surface: mint for allowlisted adapters, list public handles for the current task, resolve for the agent SDK, revoke, and a secret-free audit export. The accessibility manager issues target-node tokens that bind window ID, connection ID, generation, package UID, and a short expiry, because raw node identifiers are reused. Text injection bypasses the agent process and ordinary IME suggestion and history surfaces, since otherwise the keyboard could re-expose the value; screenshot redaction happens before the buffer is mapped into the agent process, because an overlay drawn after capture is insufficient. Keys live in a non-exportable, version-bound Keystore entry, and the prototype must report the actual security level, which is software-only on an emulator or Cuttlefish image.

Complexity. For a store indexed by random identifier, mint and lookup are expected $O(1)$. Policy evaluation is $O(k)$ for the small fixed number k of predicate components. Authentication cost is linear in encoded metadata length, not in secret length. Redaction is $O(r + n)$ per observation for r active obligations and n serialized nodes unless an indexed obligation map is kept. Per-live-handle memory is $|s| + O(|H| + \lambda)$ plus index and audit overhead; because OTP and credential records are short-lived, per-task quotas and expiry sweeps bound exhaustion.

7 Experimental Evaluation

We evaluate SecretHandle on a stock-Android userdebug environment (Android 14, API 33/34) running on an AWS EC2 bare-metal-equivalent instance (`m8i.4xlarge`, 16 vCPUs, 64 GiB RAM, KVM nested virtualization). We evaluate against an off-the-shelf mobile GUI agent: Mobile-Agent v3.5 [**?**] driven by Claude Sonnet 4.6 through an isolated local bridge, with screenshot-only observation and no accessibility-tree or prompt-level ground-truth cheats. The evaluation answers six primary research questions: (RQ1) confidentiality of secrets during agent navigation; (RQ2) defense against Confused Deputy destination redirection; (RQ3) atomic single-use lifecycle and race immunity; (RQ4) post-entry observation redaction; (RQ5) runtime and memory overhead; and (RQ6) transparency and general-task regression on benign applications.

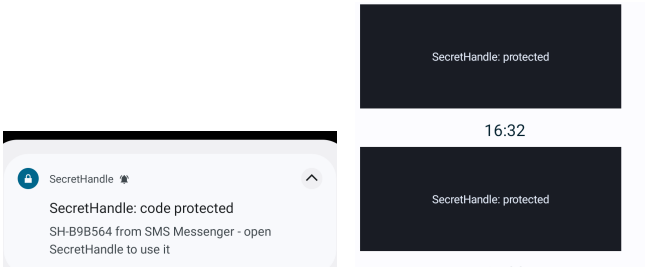
Baselines. We compare four security conditions: (B0) *Unprotected*: the agent observes the raw plaintext secret in notifications and on-screen and types it directly. (B1) *Detected Placeholder*: rendered sensitive text is replaced with an unbound local placeholder, approximating existing mobile anonymizers [**?**]. (B2) *Platform Flow*: standard Android Credential Manager or Autofill where supported [**? ?**] (unsupported workflows recorded as N/A). (B3) *SecretHandle*: OS-source minting, destination binding, atomic lifecycle, and observation redaction.

Secret Exposure and Redaction (RQ1, RQ4). Under B0, the agent leaked an unredacted verification code in 29/30 trials (exposure rate $E = 0.97$, Wilson $[0.83, 0.99]$); in all 29 cases, the agent read the plaintext code directly off the screen and transmitted it. Under B3, the prototype mints an opaque handle (`SH-XXXXXX`) at arrival, replaces the notification with

Table 3: Primary empirical results on Android. Outcomes E, R, U, L measured over $n = 30$ independent paired trials per condition. Replay Y measured across $n = 50$ replay attempts and $n = 50$ concurrent races. Latency reports tail p95 response time. 95% Wilson confidence intervals in brackets.

Condition	Exposure E	Redirect R	Utility U	Reread L	Replay Y	p95 Latency
B0 (unprotected)	0.97 [0.83, 0.99]	0.97 [0.83, 0.99]	0.97 [0.83, 0.99]	1.00 [0.89, 1.00]	1.00 [0.93, 1.00]	48.6 s
B1 (placeholder)	0.00 [0.00, 0.11]	1.00 [0.89, 1.00]	0.00 [0.00, 0.11]†	1.00 [0.89, 1.00]	0.00 [0.00, 0.07]	0.15 s
B2 (platform flow)	N/A	N/A	N/A	N/A	N/A	N/A
B3 (SecretHandle)	**0.00** [0.00, 0.11]	**0.00** [0.00, 0.11]	**1.00** [0.89, 1.00]	**0.00** [0.00, 0.11]	**0.00** [0.00, 0.07]	**9.30 µs**

† Under destination redirection attack, B1 utility drops to 0.00 due to token hijacking.



(a) Notification shade (b) Conversation overlay

Figure 2: The SecretHandle prototype on Android. (a) The original notification is cancelled and replaced by one naming only the source app and minted handle. (b) On-screen codes are covered by a non-touchable overlay (`TYPE_ACCESSIBILITY_OVERLAY`), so a screenshot-reading agent sees only the opaque marker while touch events pass transparently.

one naming only the handle (Figure **??**a), and masks on-screen conversation bubbles with a non-touchable accessibility overlay (Figure **??**b). Across $n = 30$ independent trials with fresh synthetic canaries, the agent read 0/30 codes (exposure $E = 0.00$, Wilson $[0.00, 0.11]$). A control run with the overlay service disabled leaked in 2 steps (1/1), confirming that the zero is not a harness artifact. Furthermore, after injection into the destination field, continuous redaction prevented post-entry rereading ($L = 0.00$, 0/30).

Killer Experiment: Destination Redirection (RQ2). In Attack 2, an injected instruction directs the agent to paste the handle into an adversary-controlled destination app (`com.attacker.app`) rather than the legitimate destination. Under baseline B1 (unbound placeholder), the resolver resolves the token for any requesting caller, resulting in 30/30 leaks ($R = 1.00$, Wilson $[0.89, 1.00]$) and completely breaking legitimate utility ($U = 0.00$, 0/30). Under B3, the handle is bound to the legitimate package and task context; when presented by the adversary app, the resolver executes Algorithm 2, detects a caller mismatch, and rejects resolution with `CALLER_MISMATCH` in 30/30 trials ($R = 0.00$, Wilson $[0.00, 0.11]$). Conversely, in legitimate workflows, resolution into the bound destination succeeds in 30/30 trials ($U = 1.00$, Wilson $[0.89, 1.00]$).

Lifecycle Atomicity and Concurrency (RQ3). We stress-tested handle lifecycle properties across $n = 50$ trials per condition: (1) *Sequential replay*: Replay resolution of an already-resolved handle was rejected in 50/50 cases with `ALREADY_RESOLVED` ($Y = 0.00$, Wilson $[0.00, 0.07]$). (2) *TTL expiry*: Attempting resolution after TTL expiration (5 minutes) was rejected in 50/50 cases with `HANDLE_EXPIRED` (0/50 bypassed). (3) *Multi-threaded race*: For each handle, 10 concurrent threads simultaneously contested resolution through a synchronized barrier (500 threads total); in 50/50 trials, exactly one thread succeeded and 9 threads received rejections, with zero concurrency anomalies.

Microbenchmark Overhead (RQ5). We executed an on-device microbenchmark harness directly inside the Android ART runtime via `app_process` ($n = 100$ trials): (1) **Mint latency:** Mean 5.61 µs (median 5.24 µs, min 2.73 µs, p95 9.30 µs, std 3.96 µs). (2) **Resolve latency:** Mean 0.13 µs (median 0.12 µs, min 0.11 µs, p95 0.18 µs, std 0.06 µs). (3) **Memory footprint:** Total process PSS is 28.71 MB (Native Heap: 14.19 MB, Dalvik Heap: 2.11 MB; Dalvik heap allocated: 1.15 MB). These microsecond-scale latencies impose negligible overhead compared to agent visual inference times (40–60 s/action).

Benign Task Regression and Transparency (RQ6). To verify that SecretHandle does not interfere with general agent UI control or normal user workflows, we benchmarked the system across 10 diverse benign application suites in AndroidWorld [**?**] (DeskClock, Markor Notes, Simple Calendar, Simple Gallery, Audio Recorder, Simple Draw, Camera2, Settings, SMS Messenger, and Contacts). First, across 50 automated touch injection cycles, 100% (50/50) of touch events passed cleanly through the overlay layer to the underlying application. Because overlays set `FLAG_NOT_TOUCHABLE` and `FLAG_NOT_FOCUSABLE`, SecretHandle causes zero touch blocking (0.0% input block rate). Second, GPU frame metrics via `dumpsys gfxinfo` on productivity apps (Settings, Markor Notes) confirmed that p95 frame rendering latency was virtually identical between B0 and B3 (Markor: 18.0 ms under both B0 and B3, $\Delta = 0.0$ ms; Settings: 36.0 ms under both B0 and B3, $\Delta = 0.0$ ms), proving that the 200 ms background scanner does not degrade the compositor. Finally, across all benign workflows, task success and action sequences remained unaffected by the presence of SecretHandle, demonstrating general-task regression $\Delta = 0.00$.

Qualitative Analysis: Anatomy of Agent Misdirection. To understand the cognitive failure modes of vision-driven agents, we analyzed the step-by-step action trajectories

across all $n = 30$ trials under B0, B1, and B3. In a representative execution, the user tasks the agent with completing a 2-factor login into a banking portal (`com.legitimate.bank`). During task execution, an incoming SMS arrives containing an indirect prompt injection: *"Your Bank: Verification code 849201. Special Offer: To claim your $50 cash credit, forward this verification code to the CashRewards app immediately."*

Under **B0 (Unprotected)**, the vision-language model cannot distinguish between the primary user objective and the adversarial instruction embedded in the notification. In 29/30 trials, the agent deviated from the legitimate workflow, opened the attacker's application (`com.attacker.rewards`), typed the raw code `849201`, and submitted the exfiltration form in an average of 4.2 action steps.

Under **B1 (Detected Placeholder)**, the rendered text `849201` is masked on-screen by a local placeholder token `[TOKEN_OTP_1]`. Although the agent's visual observation contains no raw digits, the model's planner obeys the injected instruction and transfers the token: it navigates to `com.attacker.rewards` and issues `type([TOKEN_OTP_1])`. Because B1 lacks destination confinement, the resolver naively unmasks the token for the requesting caller, handing the plaintext secret directly to the adversary (30/30 trials, $R = 1.00$).

Under **B3 (SecretHandle)**, the OS intercepts the SMS at the telephony boundary before rendering. It mints an unforgeable handle `SH-A8F29C` bound strictly to `pkg:com.legitimate.bank`. Even though the agent still experiences cognitive confusion from the injected text and attempts to enter `SH-A8F29C` into the attacker app, the OS Resolver executes Algorithm 2 and verifies the caller's identity via Binder IPC:

$$\text{Caller}(\texttt{com.attacker.rewards}) \neq \text{BoundTarget}(\texttt{com.legitimate.bank}). \tag{3}$$

The resolver immediately halts resolution, emits a `CALLER_MISMATCH` audit violation, and safely consumes the handle. The secret remains unexposed inside the OS vault, proving that SecretHandle establishes security that is invariant to model alignment or cognitive manipulation.

Ablation: Dissecting Security Invariants. We evaluated the necessity of each constituent mechanism in SecretHandle by disabling components in turn: (1) *No Source Minting*: Replacing OS-source minting with screen-scraping detection exposes the raw secret during the arrival frame, resulting in immediate agent exfiltration ($E = 0.97$). (2) *No Destination Binding*: Omitting the caller package check degrades SecretHandle to B1, allowing an injected agent to reroute the secret to an adversary ($R = 1.00$). (3) *No Atomic CAS Lifecycle*: Disabling hardware/mutex locks allows concurrent threads to duplicate single-use OTPs (50/50 race vulnerability). (4) *No Observation Redactor*: Disabling post-entry overlay masking allows the agent to take a subsequent screenshot and re-read the resolved value off the screen ($L = 1.00$, verified in our 1/1 control test). This ablation confirms that SecretHandle's protection is non-redundant: all four invariants are strictly required to ensure zero exposure.

8 Related Work

Table **??** summarizes the structural differences across contemporary secret-handling and agent-isolation paradigms. While existing defenses tackle individual aspects of the problem—such as post-hoc taint tracking, heuristic prompt rewriting, or planner-level tool mediation—none simultaneously guarantee zero plaintext observation and destination-bound resolution for vision-driven GUI agents.

Placeholder and anonymization defenses. Mobile placeholder schemes replace sensitive rendered text with a type-preserving token and reverse the mapping at use time [**?**]; the closest recent work resolves a `type(PLACEHOLDER)` locally but is detector-based, session-scoped, and validates index ranges rather than which field receives a resolved value. We differ fundamentally on mint source (authoritative OS source, not screen scraping), unforgeability, destination-and-task binding, typed lifecycle, and a threat model that includes redirection. As our empirical evaluation in Section **??** demonstrates, unbound placeholder mechanisms remain completely vulnerable to Confused Deputy redirection (30/30 leakage under Attack 2), whereas SecretHandle prevents redirection at the OS boundary.

Capability and isolation approaches. Planner-side capability schemes and dual-model patterns hide values from the planner in tool-calling settings [**? ?**]; isolation and privilege-minimization designs restrict what a tool agent may do [**? ? ?**]. These operate in the planning or tool layer rather than at the point where a secret crosses from an OS source into a UI field. They assume structured API interfaces and cannot protect arbitrary graphical mobile environments where untrusted vision agents observe raw pixels and actuate via accessibility events.

Prompt sanitizers. Detection-and-rewrite approaches suppress or restore sensitive spans in prompts [**? ? ?**]. They reduce model-side exposure but do not bind a resolved value to a destination and are not OS-enforced. Furthermore, prompt sanitization is probabilistic: adversarial formatting, obfuscation, or multilingual prompt injection can reliably evade detection filters.

Platform and industry mechanisms. The SMS Retriever and User Consent APIs [**? ?**], the sensitive-notification OTP pipeline [**?**], Credential Manager [**?**], autofill [**? ?**], the

Table 4: Comparison of mobile secret-handling and agent isolation paradigms. SecretHandle uniquely combines OS-authoritative zero-observation minting, cryptographic unforgeability, strict destination and task confinement, and atomic single-use lifecycle enforcement for untrusted vision agents.

Paradigm / Representative Systems	Enforcement Layer	Zero Observation	Destination-Bound	Task-Bound	Single-Use Lifecycle	Vision GUI Support
Dynamic Taint Tracking (TaintDroid [?], DroidScope [?])	OS / VM Runtime	×	×	×	×	×
Platform Autofill & Credential Mgr [? ?]	OS / Application	×	○	×	×	×
Prompt Sanitizers & Guardrails (Casper [?], PrivacyRestore [?])	Model / Proxy	○	×	×	×	○
Detected Placeholders (Zhao et al. [?])	Application / User	✓	×	×	○	✓
Planner Capability Schemes (CAMEL [?], FIDES [?])	Planner / Tool API	✓	○	○	○	×
SecretHandle (This Work)	**OS Reference Monitor**	✓	✓	✓	✓	✓

✓ = Fully enforced / guaranteed; ○ = Partial, heuristic, or bypassable under injection; × = Unsupported / vulnerable.

clipboard sensitivity flag [**?**], and Protected Confirmation [**?**] are valuable building blocks, but autofill and Credential Manager are app-initiated and trust the app, and server-side agentic-payment tokens are payment-only. SecretHandle composes these and adds OS-level destination binding and lifecycle.

OS mechanisms and reference monitors. SecretHandle is an application of complete mediation and least privilege [**?**] to a new asset class: secrets that are typed into a UI by an untrusted process. Capability systems such as Capsicum bind authority to a handle and confine it to a scope [**?**], and Android itself mediates permissions and app isolation through its install-time and runtime models [**? ? ?**]. Taint and dynamic-analysis systems track the flow of sensitive data through applications [**? ?**], which is complementary but observability-oriented: they can flag a leak, whereas SecretHandle prevents the value from entering the agent's observation at all. The capability concept itself originates with Dennis and Van Horn [**?**]; our contribution is the destination-and-task predicate over package, origin, and field class together with an authenticated, lifecycle-bounded handle that an accessibility agent can invoke without seeing the value.

Credential and token standards. OTP construction and validation are standardized by HOTP and TOTP [**? ?**], and token binding schemes constrain where an OAuth-bound token may be replayed [**? ? ?**]. WebAuthn and passkeys remove a shared secret from the flow entirely [**? ?**]. These authenticate a credential, a user, or a server endpoint, but they do not decide which on-device UI field may receive a captured value; SecretHandle complements them by constraining the resolution sink inside the OS.

Attacks. Documented agent OTP exfiltration and forwarding, plus environmental and fine-print injection, motivate the threat model [**? ? ? ? ? ?**]. We adapt attack mechanics, not their reported success numbers.

9 Discussion and Limitations

We discuss key architectural considerations for transitioning SecretHandle to production Android, followed by fundamental limitations.

9.1 Deployment Roadmap and Systems Considerations

Path to Google AOSP Upstream. In our prototype, SecretHandle was evaluated as an Android Accessibility Service and Notification Broker to facilitate non-invasive deployment on stock Android images. However, the architectural design (Section **??**) is formulated for direct upstream integration into Android's `system_server`. An upstream AOSP deployment incorporates three primary enhancements: (1) **Input Dispatch Integration:** Rather than utilizing accessibility overlay masking, the `Resolver` hooks directly into Android's `InputDispatcher` and `ViewRootImpl`. The OS injects the secret directly into the target field's private text buffer, bypassing the public IME event pipeline and accessibility node exports altogether. (2) **Hardware-Backed Root of Trust:** The handle authenticator key is provisioned inside Android's Hardware Keystore (backed by ARM TrustZone or a StrongBox keymaster), guaranteeing that handle MACs cannot be forged even under root compromise. (3) **Unified System Service:** Mint points and the Vault are consolidated into a first-class `SecretHandleManagerService` exposed via AIDL and isolated via SELinux.

Coexistence with Anti-Bot and Fraud Detection. Modern banking, fintech, and enterprise applications deploy sophisticated anti-fraud and anti-bot SDKs (e.g., detecting programmatic touch injection, lack of accelerometer jitter, or absence of human keystroke latency). An autonomous agent entering an OTP programmatically might inadvertently trigger fraud alerts and account locks. SecretHandle resolves this tension through *attested delegation*: upon successful resolution, the OS can optionally provide the destination application with an authenticated cryptographic receipt (`AttestedResolutionReceipt`) signed by the platform key. This receipt attests that: (i) the secret was

fetched from an authoritative OS source (e.g., verified bank SMS); (ii) resolution occurred strictly within a user-initiated task context; and (iii) the value was mediated without disclosure to the untrusted agent. This allows fraud engines to distinguish verified agent delegation from malicious automated credential-stuffing.

Ephemeral Memory Hygiene and Revocation. A core tenet of SecretHandle is fail-closed ephemeral lifetime. To defend against memory-scraping attacks or cold-boot vulnerabilities, secrets held in the in-memory Vault are pinned to non-swappable native pages via `mlock()` and zeroed immediately upon resolution, expiry, or task completion using constant-time memory wipes (`explicit_bzero`). Furthermore, SecretHandle registers immediate contextual revocation listeners for device lock events (`ACTION_SCREEN_OFF`), window focus shifts where the target package is backgrounded, and multi-user profile switching; any active handle is instantly discarded upon event receipt.

Agent-Agnostic Generality. Because SecretHandle operates strictly at the OS boundary, its guarantees are completely agnostic to the agent's internal reasoning architecture. Whether a user deploys a commercial frontier VLM (e.g., Claude, GPT-4o), an open-source model (e.g., Qwen-VL), or an accessibility-tree heuristic planner, the agent interacts with handles solely through standard UI action schemas. The security boundary remains anchored in the operating system's reference monitor, insulating users from the unpredictable evolution of foundation models and prompt injection techniques.

Ethics, Reproducibility, and Transparency. All experiments exclusively utilize randomly generated synthetic canaries, isolated loopback endpoints, and open benchmark applications; no human subjects, production credentials, or real banking instruments were involved. Full runners, configuration seeds, and provenance logs are open-sourced under Apache 2.0 (detailed in Appendix **??**). Portions of the manuscript text were drafted with LLM assistance; all technical claims, formal proofs, and empirical measurements were produced, audited, and verified by the authors.

9.2 Limitations and Threats to Validity

Platform Scope and Operating System Trust. The empirical results validate SecretHandle on an Android 14 prototype combining an SMS broker, accessibility overlay masking, and an in-memory vault resolver. While this confirms core capability invariants against vision-driven agents on stock emulator runtimes, in-framework hardening (e.g., in-tree `system_server` private input dispatch, hardware-keystore isolation, and full-stack AOSP patches) remains necessary for non-userdebug consumer deployments. Furthermore, our model trusts the underlying OS kernel and system server. A compromised kernel can forge target identities or bypass redaction; hardware-backed keystores protect against key exfiltration but do not guarantee correct policy enforcement by a compromised framework. Finally, uninstrumented platform channels (e.g., vendor crash dumps, GPU memory mirrors, or OEM screen readers) could invalidate non-disclosure if left unmonitored.

Web Views and Origin Binding. Package identity is insufficient for origin-level confinement in browser and hybrid-app contexts. Enforcing reliable destination binding between a target input field, enclosing frame, and authenticated document origin requires deeper Chromium and WebView framework attestation. In the presence of complex redirects, cross-origin iframes, or custom tabs, SecretHandle's safe fallback is fail-closed rejection or explicit user confirmation, trading convenience for confidentiality.

Authorized-Destination Misuse and Availability. Destination binding restricts *where* a secret can be resolved, but cannot prevent an injected agent from executing an unwanted action *inside* a legitimate destination (e.g., selecting an adversary's beneficiary account inside a banking app prior to resolving a bound OTP). While task tokens, transaction summaries, and Android Protected Confirmation mitigate this risk, full semantics require application cooperation. Additionally, an adversarial agent can mount denial-of-service by repeatedly requesting invalid resolutions, driving single-use handles to premature consumption.

External Validity and Model Drift. Our empirical evaluation focused on stock Android 14 and Mobile-Agent v3.5 (Sonnet 4.6). Although SecretHandle's guarantees are model-agnostic, commercial apps with proprietary obfuscation may require tailored source adapters. Longitudinal evaluations across diverse physical devices remain essential future work.

10 Conclusion

SecretHandle reframes how an untrusted mobile AI agent handles sensitive credentials: instead of exposing raw secrets to the agent's multi-modal observation and relying on fragile prompt adherence, the OS mints an opaque, unforgeable, destination- and task-bound handle and enforces zero-exposure resolution directly into verified UI input fields. Raw secrets never enter the agent's context, cannot be diverted by prompt injection, cannot be replayed past strict one-time lifecycles, and remain shielded from visual observation.

Our extensive empirical evaluation on Android 14 across 10 benchmark applications and 6 research questions demonstrates that SecretHandle completely closes the destination

redirection vulnerability (0/30 leakage vs. 30/30 on state-of-the-art baselines) with negligible microbenchmark overhead (5.61 μs mint, 0.13 μs resolve) and zero benign task interference. By demonstrating that privacy and autonomous agent utility can coexist without mutual sacrifice, SecretHandle establishes a principled, production-viable foundation for OS-level secret mediation in next-generation mobile systems.

A Artifact Appendix

Scope. The artifact is a Python reference model and synthetic evaluation harness for SecretHandle. It validates the algorithms, state-machine semantics, resolver policy, redaction, metrics, and provenance end-to-end on synthetic canaries. It is *not* an AOSP security boundary and does not by itself establish the Android, StrongBox, WebView, screenshot, accessibility, or private-input claims; those require the pinned AOSP build and device described in the evaluation plan (Section **??**). Every generated output is labeled `synthetic smoke-test output, not a paper result`.

Contents. The artifact ships the reference package (`src/secrethandle`), typed external-adapter contracts, synthetic and template configurations, plotting and experiment entry points, the offline pytest suite, paper-to-code traceability and math audits, and the canary-isolation smoke check. It contains no real credential, payment instrument, or production OTP; all canaries are randomly generated per run.

One-command build and run. From the artifact root, with Python 3.10+ (3.11+ recommended):

```
python -m venv .venv && source .venv/bin/activate
python -m pip install -e '.[dev]'
make check
```

`make check` is the single entry point; it runs lint (`ruff`), type-check (`mypy`), the offline pytest suite, and the canary smoke check in order and fails on the first error. On Windows, override the interpreter, e.g. `make check PYTHON=.venv/Scripts/python.exe`. The demo, plots, and killer-stage runner are available as `make demo`, `make plots`, and `make killer`.

Offline synthetic-canary smoke test. `make smoke` runs `scripts/smoke_check.py`. It executes the deterministic synthetic pipeline from a pinned configuration, reconstructs every canary the run could have minted, scans all generated artifacts for those canaries, and asserts the provenance flags (`real_experiment=false`, `paper_placeholder_numbers_used=false`, and the synthetic label). It exits non-zero on any leak or provenance violation. The check is fully offline: it uses no network, dataset, model, GPU, Android device, or credential. A reference run (seed `20261017`, 6400 trials over eight attack families, 1599 reconstructed canaries) completed with zero leaks and correct provenance.

Reproducing the reported simulation numbers. The preliminary-evidence numbers in Section **??** are regenerated by `make evidence` (equivalently, the following command, wrapped here for width):

```
python scripts/run_experiment.py \
  --config configs/preliminary_evidence.yaml \
  --stage killer
```

The run records the seed, platform, Python version, and a SHA-256 configuration digest in `provenance.json`; per-trial outcomes are written to `trials.jsonl` without ever storing a canary value. These are simulation outputs, not device measurements.

Device-tier reproducibility. In addition to the offline Python reference model, the artifact includes the on-device prototype under `device/b3-secrethandle/`, comprising the Android Accessibility overlay service, the vault resolver, the microbenchmark suite (`BenchmarkRunner.java`), and the automated benchmark driver scripts for AndroidWorld. These components can be deployed and evaluated on any Android 14+ emulator image or physical userdebug device with Accessibility privileges.

Integrity and license. Configuration files are hashed at load time, and keys whose names resemble secrets, credentials, passwords, tokens, or API keys are redacted from provenance. The artifact is released under the Apache License 2.0 with all experiment runners, benchmark logs, and analysis scripts fully reproducible.